

TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



Lab Report of
Introduction to Cloud Computing

A Lab Report Submitted as required for Cloud Computing

Bachelor of Science in Computer Science & Information Technology (B.SC. CSIT)

8th Semester of Tribhuvan University, Nepal

Submitted By:
Aayush Basnet
29131/078

Submitted to:
Suvash Khadka

March 13, 2026

Name: Aayush Basnet

Symbol: 29131

Introduction to Cloud Computing

SN	Name of Practical	Date of Practical	Signature
1.	Virtualization using VMWare and Creation of Multiple Virtual Machines		
2.	Modification of Virtual Machine Resources		
3.	IP Address Identification and Network Connectivity Testing		
4.	Performing security drills and audit using Nmap		
5.	Implementation of MapReduce Program		
6.	Capturing Packets using Wireshark		
7.	Using Site24*7 monitor all necessary parameters for the MBM college.		

Q7) Using Site24*7 monitor all necessary parameters for the MBM college. You should preserve the output, for each control and monitoring step performed.

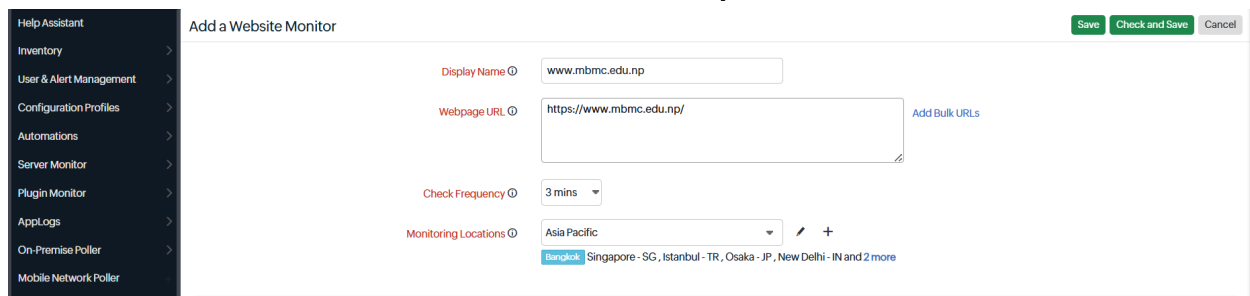
The purpose of this lab was to gain practical experience in setting up and utilizing monitoring tools to ensure the optimal performance and reliability of IT infrastructure. Specifically, we focused on using Site24x7 to monitor a Linux-based virtual machine (VM) running Ubuntu on VirtualBox. By performing this lab, we aimed to understand how to install monitoring agents, configure monitoring parameters, and analyze the collected data through the Site24x7 dashboard. This hands-on experience is crucial for IT professionals to proactively manage and troubleshoot network and system issues.

Objectives

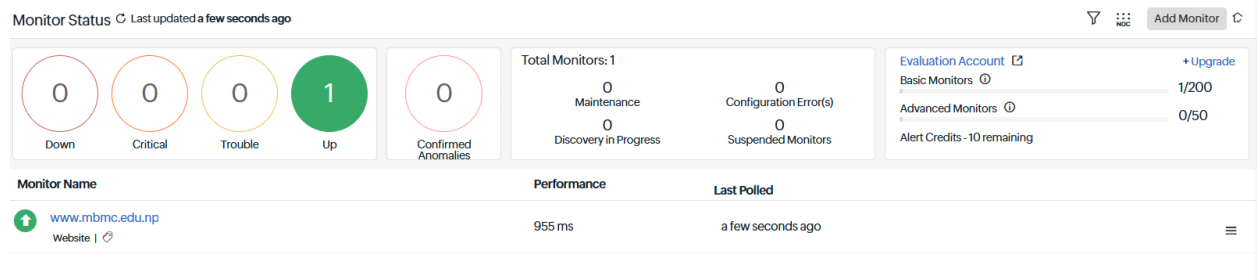
The objectives of this lab are as follows:

- Successfully install the Site24x7 monitoring agent on an Ubuntu VM to enable data collection.
- Configure Site24x7 to monitor essential system metrics such as CPU usage, memory usage, disk space, and network activity.
- Use the Site24x7 dashboard to review collected data, generate reports, and identify potential issues or performance bottlenecks.

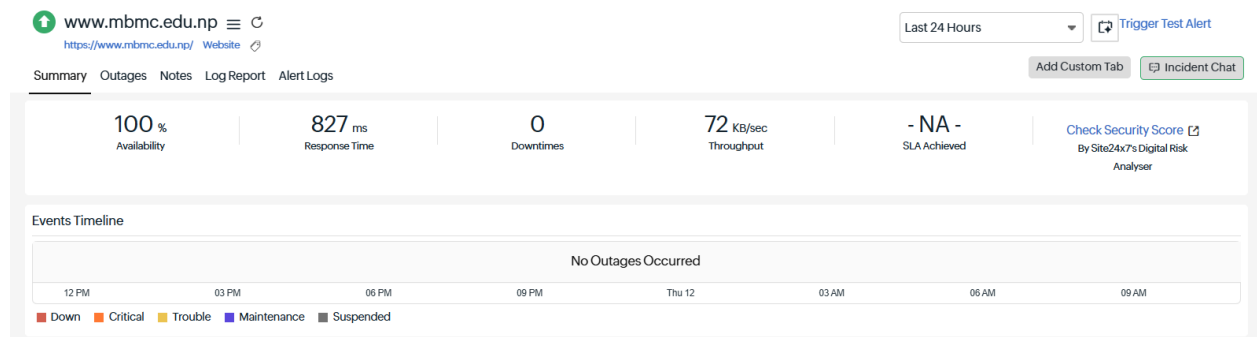
Access the dashboard of site24*7 and review the reports



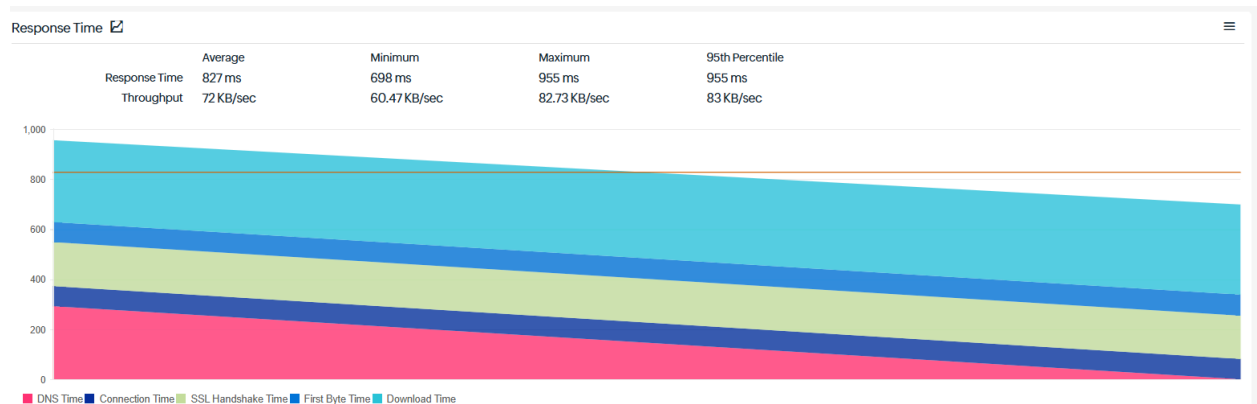
The screenshot shows the 'Add a Website Monitor' interface in the Site24x7 dashboard. On the left is a dark sidebar with navigation links: Help Assistant, Inventory, User & Alert Management, Configuration Profiles, Automations, Server Monitor, Plugin Monitor, AppLogs, On-Premise Poller, and Mobile Network Poller. The main form area has the title 'Add a Website Monitor' and three buttons at the top right: 'Save', 'Check and Save', and 'Cancel'. The form fields are as follows: 'Display Name' with the value 'www.mbm.edu.np'; 'Webpage URL' with the value 'https://www.mbm.edu.np/' and a blue 'Add Bulk URLs' link; 'Check Frequency' with a dropdown set to '3 mins'; and 'Monitoring Locations' with a dropdown set to 'Asia Pacific' and a list of locations below: 'Bangkok', 'Singapore - SG', 'Istanbul - TR', 'Osaka - JP', 'New Delhi - IN' and '2 more'. Each field has a small circular icon with an 'i' for information.



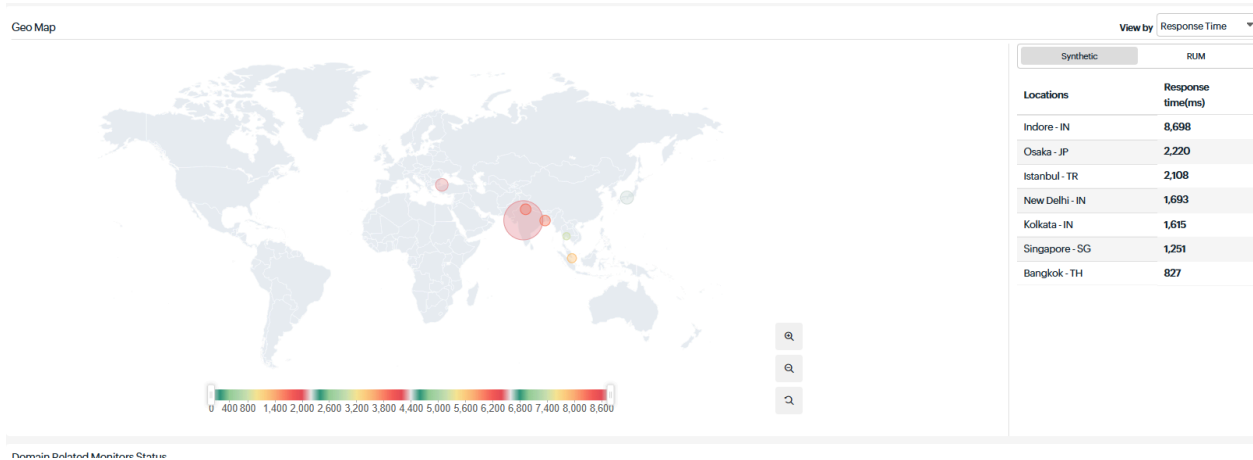
Analyzing Parameters



Response Time



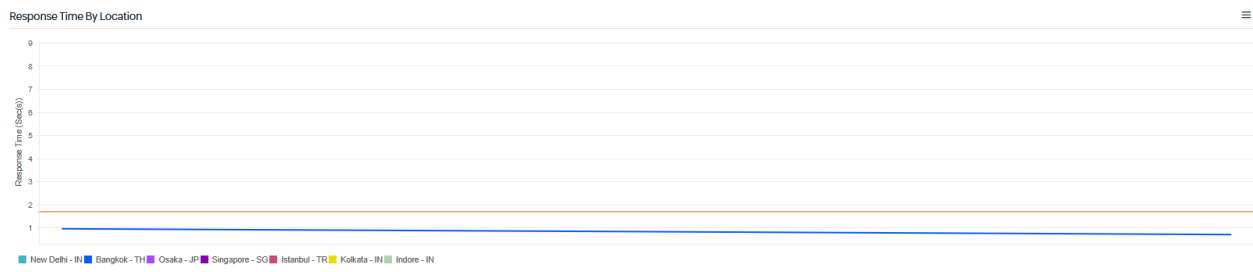
Geo Map



Availability and Response Time by Location

Location	Availability (%)	Down Duration	Downtimes	Last Downtime	Response Time (ms)
Bangkok - TH	100	0 Mins 0 Secs	0	-	1460817582344827
Singapore - SG	100	0 Mins 0 Secs	0	-	20656139687721251
Istanbul - TR	100	0 Mins 0 Secs	0	-	131056347199130082108
Osaka - JP	100	0 Mins 0 Secs	0	-	1543143428002220
New Delhi - IN	100	0 Mins 0 Secs	0	-	330167812471693
Indore - IN	100	0 Mins 0 Secs	0	-	1265179166348698
Kolkata - IN	100	0 Mins 0 Secs	0	-	36891110691615

Response Time By Location



Global Industry Average

Global Industry Average



Response Time



Availability



Here's what you should add next to make the most of your Site24x7 account.

Server Monitoring

Monitoring your back-end servers is equally important to ensure your internet services are up and running 24x7. Install an agent and keep a check on your server performance with 60+ performance metrics.

[Learn more](#)

[Add a Server Monitor](#)



Q1) Perform virtualization using VMWare and following virtual machines: Ubuntu, Kali

Virtualization is a fundamental concept in cloud computing that allows multiple operating systems to run on a single physical machine. This is achieved using a hypervisor such as VMware. Virtual machines share hardware resources like CPU, RAM, and storage while functioning independently. Virtualization improves resource utilization, scalability, and system management which are core characteristics of cloud computing platforms.

Key Technical Terms

Virtualization – Technology that allows multiple operating systems to run on a single hardware system.

Virtual Machine (VM) – A software-based computer system that runs an operating system just like a physical computer.

Hypervisor – Software that manages virtual machines (e.g., VMware).

ISO Image – A disk image file used to install an operating system.

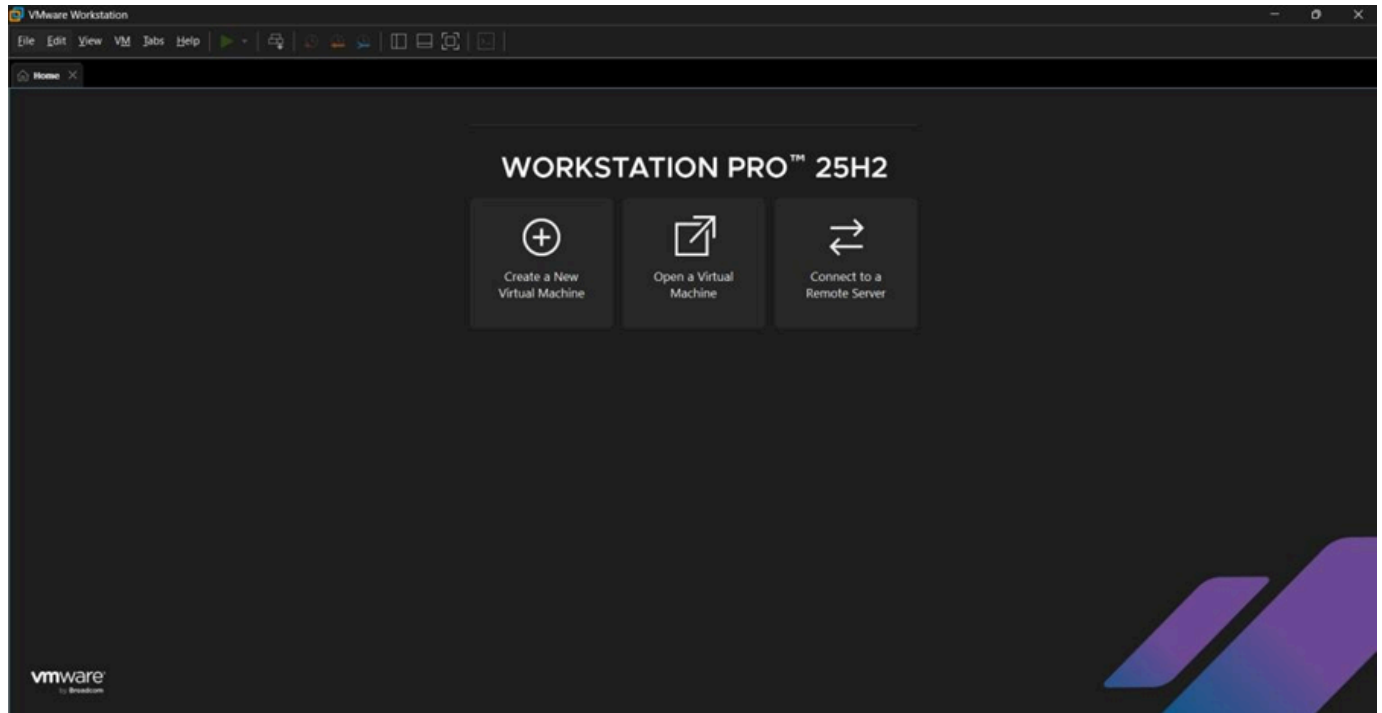
Resource Allocation – Assigning CPU, RAM, and storage to virtual machines.

Procedure

Step 1: Install VMware Workstation.

1. Download and install VMware Workstation from the official VMware website:

VMware Workstation.



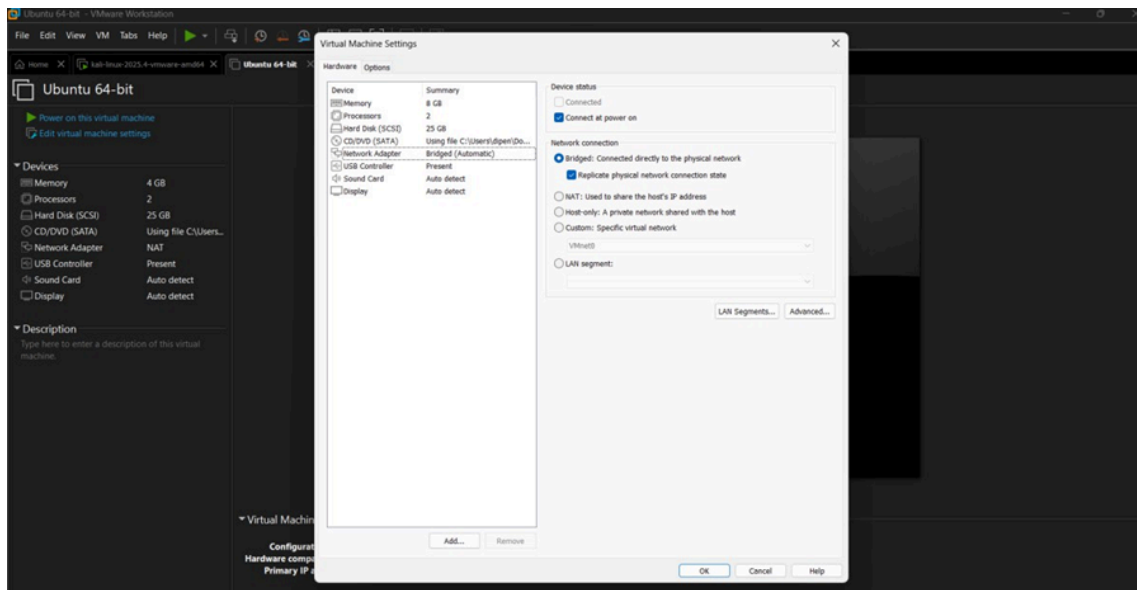
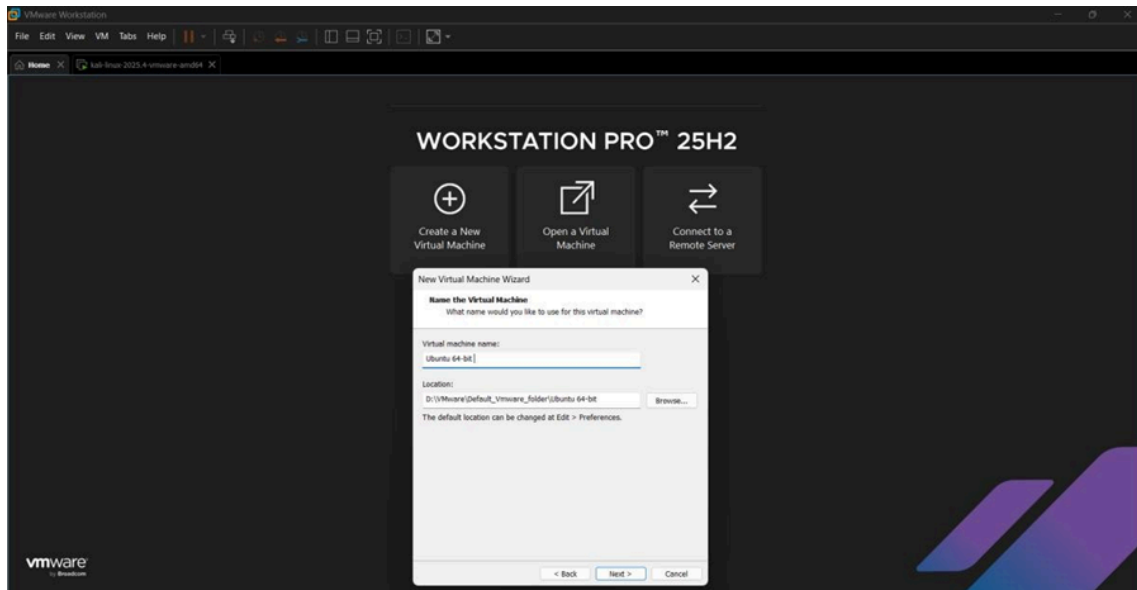
Step 2: Create a Virtual Machine(VM) for Ubuntu OS.

1. Launch VMware Workstation:
 - Open VMware Workstation or VMware Player.
2. Create a New VM:
 - Click "Create a New Virtual Machine".
 - Choose "Typical (recommended)" and click Next.
3. Select the OS Type:
 - Select "Installer disc image file (iso)" and browse to the Ubuntu ISO file.
 - Select Ubuntu under the operating system options.
4. Configure the VM Hardware:
 - Memory: Assign at least 2 GB RAM (or more based on your machine's capacity).
 - Processors: Assign at least 1 processor (2 cores if your system allows).
 - Storage: Assign at least 20 GB of virtual hard drive storage.
 - Select "Create a new virtual disk" and set the disk size.
5. Complete VM Creation:

- Name your VM (e.g., "Ubuntu_VM").
- Click Finish to complete the VM creation.

6. Start the VM:

- Click Power On to boot the VM from the ISO.
- The system will boot into Ubuntu installation. Follow the on-screen instructions to install Ubuntu.





- In VMware, go to File > New Virtual Machine and choose "Typical".
- Click Next.

2. Select the OS Type:

- Select "Installer disc image file (iso)" and browse to the Kali Linux ISO file.
- Select Linux and then Debian 64-bit (since Kali is based on Debian).

3. Configure the VM Hardware:

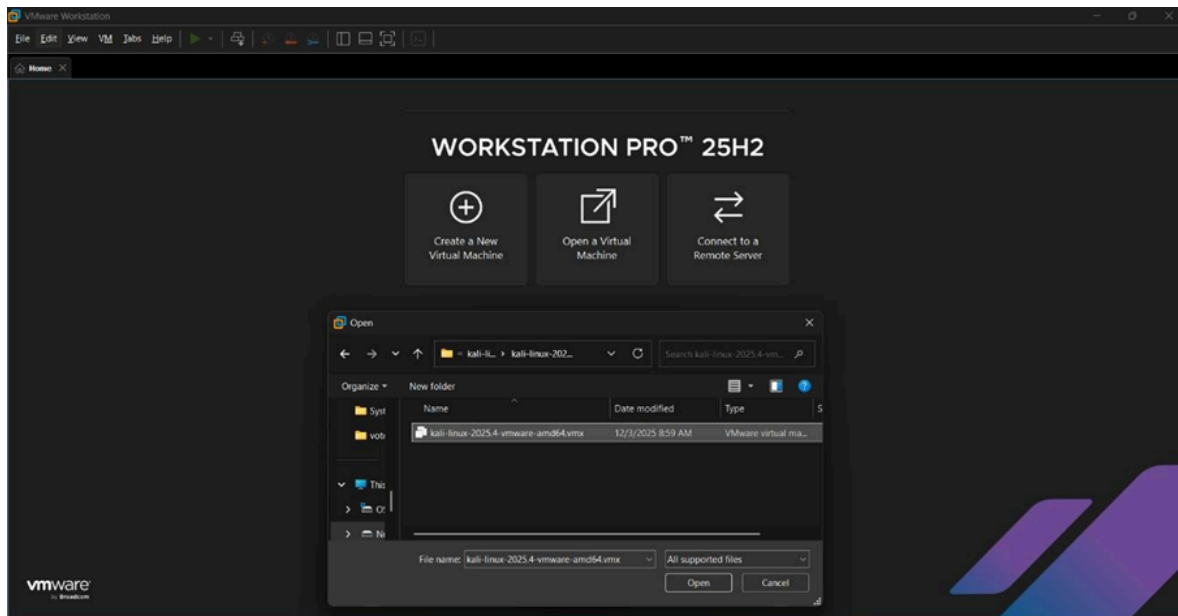
- Memory: Assign at least 2 GB RAM (or more based on your machine's capacity).
- Processors: Assign at least 1 processor (2 cores if your system allows).
- Storage: Assign at least 20 GB of virtual hard drive storage.
 - Select "Create a new virtual disk" and set the disk size.

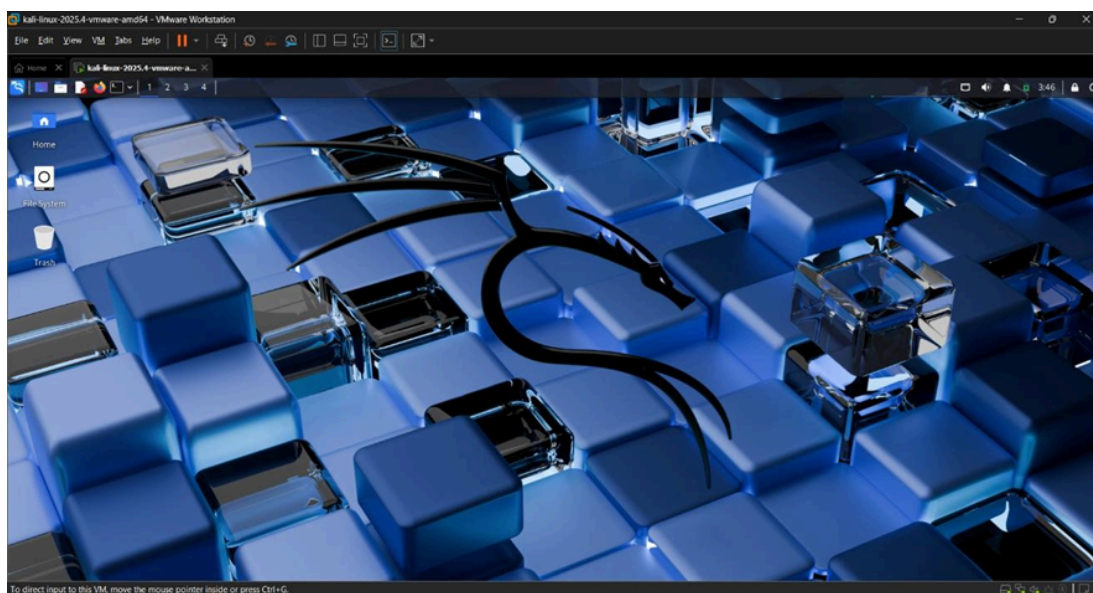
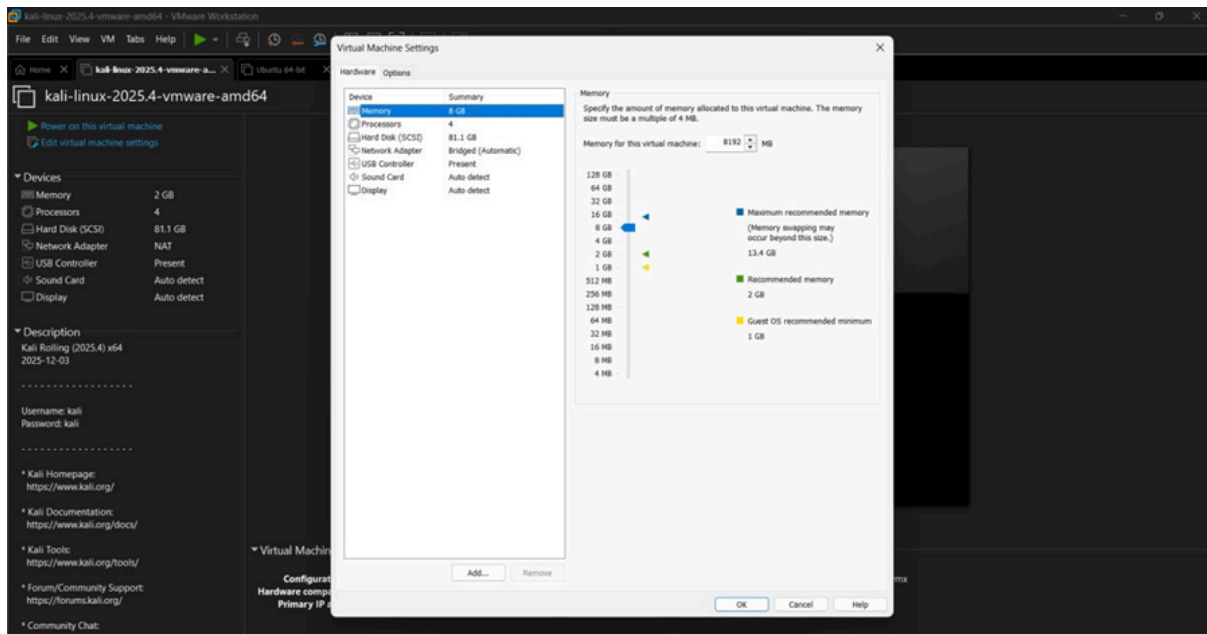
4. Complete VM Creation:

- Name your VM (e.g., "Kali_Linux_VM").
- Click Finish to complete the VM creation.

5. Start the VM:

- Click Power On to boot the VM from the Kali Linux ISO.
- Follow the on-screen instructions to install Kali Linux.





Q2) For the VM, change the RAM to more than 8GB and hard disk size more than 20GB, also write the observation for it.

Virtualization technology allows users to create and manage multiple virtual machines on a single physical computer. One of the major advantages of virtualization is the ability to modify hardware resources such as memory (RAM) and storage without affecting the host machine. Virtual machine managers like VMware Workstation allow administrators to dynamically adjust resources according to system requirements. Increasing RAM and disk space improves the performance and storage capacity of virtual machines

Materials and Equipment

- Computer or laptop with virtualization support
- **VMware Workstation** installed
- Previously created virtual machines:

-Ubuntu Virtual Machine

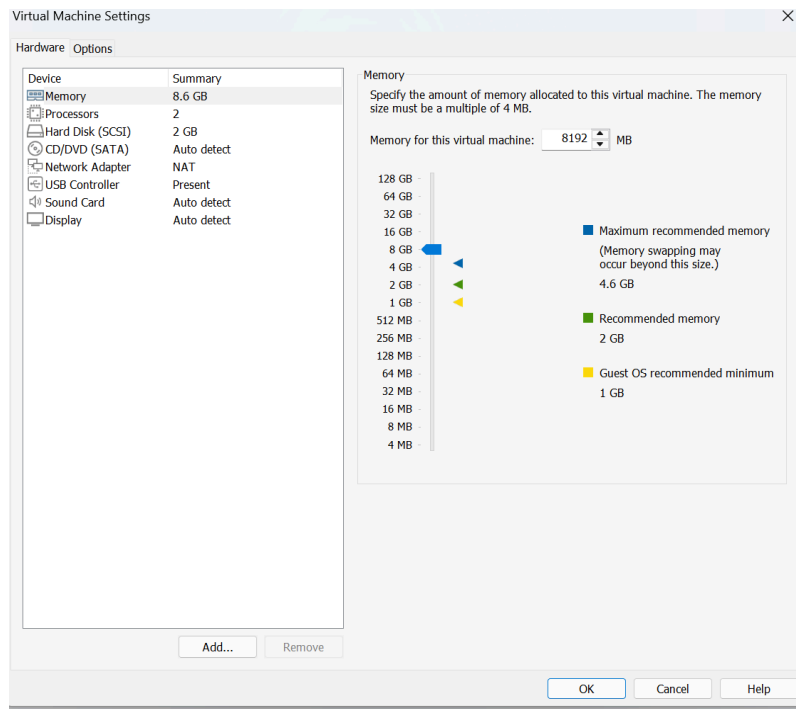
-Kali Linux Virtual Machine

- Operating system already installed in the VMs

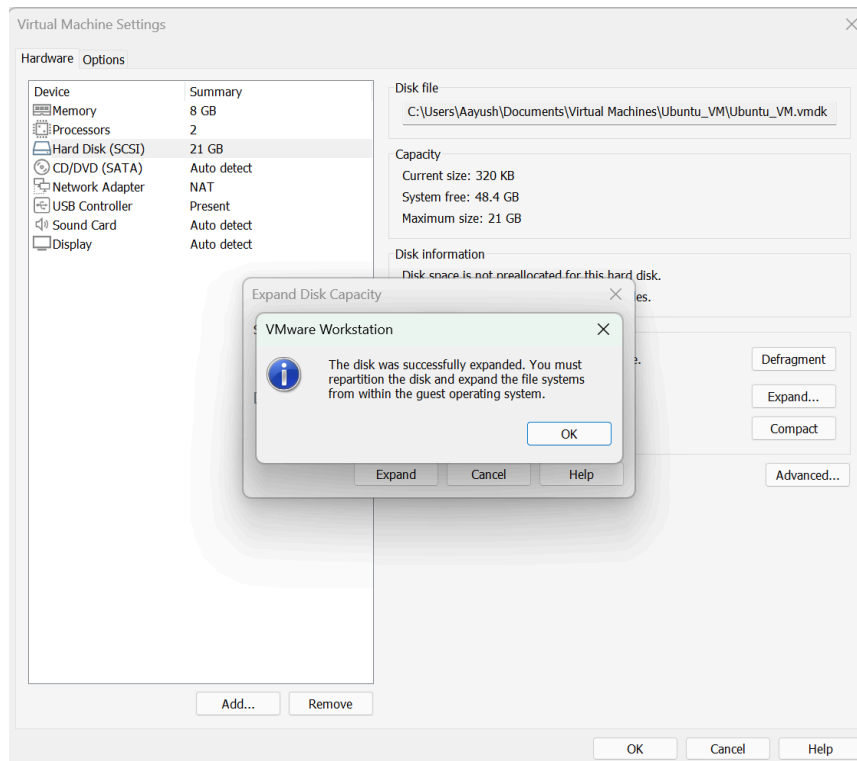
Procedure

1. Open VMware Workstation on the host computer.
2. Select the first virtual machine (Ubuntu VM) from the VMware library.
3. Click on Edit Virtual Machine Settings.
4. Select the Memory option.
5. Increase the RAM size to more than 8 GB.
6. Click on Hard Disk settings.
7. Increase the disk size to more than 20 GB.
8. Click OK to save the changes.
9. Repeat the same steps for the second virtual machine (Kali Linux VM).
10. Start both virtual machines and verify the updated hardware configuration.

Changing RAM to more than 8GB



Expanding hard disk size more than 20GB



Q3) From all VMs, show the ip address of all VMs. then ping IP address of VM.

In a virtualized environment, each virtual machine connected to a network receives a unique IP address. This IP address allows machines to communicate with each other within the network. Network testing tools such as the ping command help verify connectivity between systems. This experiment demonstrates how to identify IP addresses and test communication between two virtual machines.

Objectives

- To find the IP address of virtual machines.
- To test connectivity between two virtual machines using the ping command.

Procedure

1. Start both virtual machines in VMware.
2. In the Ubuntu VM, open the terminal.
3. Type the following command to find the IP address:

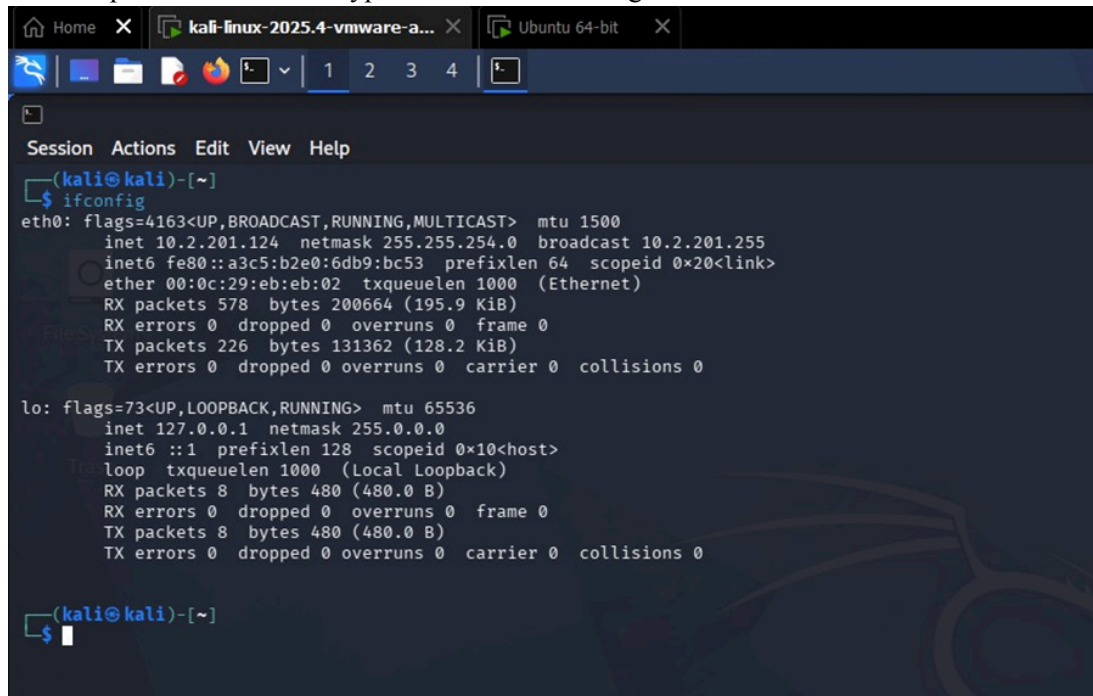
Ifconfig or ip a

4. Note the IP address displayed.
5. Open the second VM(kali Linux or Windows).
6. Open terminal or Command Prompt.
7. Use the ping command to test connectivity:

ping <IP address of other VM>

8. Observe whether packets are successfully transmitted and received.

1. Find out IP address of Kali Linux VM
Opened terminal and typed command “ifconfig”.



The screenshot shows a Kali Linux terminal window with the following content:

```
Session Actions Edit View Help
(kali@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.2.201.124 netmask 255.255.254.0 broadcast 10.2.201.255
    inet6 fe80::a3c5:b2e0:6db9:bc53 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:eb:eb:02 txqueuelen 1000 (Ethernet)
    RX packets 578 bytes 200664 (195.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 226 bytes 131362 (128.2 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@kali)-[~]
$
```


Q5) Write a program for map reduction in any of the programming platform

MapReduce is a programming model and processing technique for handling and generating large data sets with a distributed algorithm on a cluster. Developed by Google, it allows for the processing of massive amounts of data in a parallel, distributed manner across many computers.

Components of MapReduce:

Map Function:

Input: The input is typically a large data set divided into smaller chunks.

Processing: The Map function processes these chunks independently and in parallel. It takes a pair of input key-value pairs and produces a set of intermediate key-value pairs.

Output: The output of the Map function is an intermediate data set, which consists of key-value pairs that are grouped by key.

Shuffle and Sort:

Shuffling: This phase is responsible for transferring the intermediate key-value pairs produced by the Map function to the Reducers. It ensures that all values associated with the same key are sent to the same Reducer.

Sorting: Before the data is sent to the Reducer, it is sorted by the intermediate key. This helps in efficient grouping of values by key during the reduce phase.

Reduce Function: Input: The input to the Reduce function is the grouped intermediate data. Each group consists of a unique key and a list of values associated with that key.

Processing: The Reduce function processes each group independently. It takes the intermediate key and its associated list of values and combines them to produce a smaller set of values. Output: The output of the Reduce function is the final output of the MapReduce job. It usually consists of aggregated or summarized results.

WAP of map reduce to Word Count

```
MapReduceWordCount.java X
MapReduceWordCount.java > Language Support for Java(TM) by Red Hat > MapReduceWordCount > map(String)
1  import java.util.*;
2
3  public class MapReduceWordCount {
4
5      public static List<Map.Entry<String, Integer>> map(String input) {
6          List<Map.Entry<String, Integer>> mappedData = new ArrayList<>();
7          String[] words = input.split(regex: " ");
8
9          for (String word : words) {
10             mappedData.add(new AbstractMap.SimpleEntry<>(word, value: 1));
11         }
12
13         return mappedData;
14     }
15
16     public static Map<String, Integer> reduce(List<Map.Entry<String, Integer>> mappedData) {
17         Map<String, Integer> reducedData = new HashMap<>();
18
19         for (Map.Entry<String, Integer> entry : mappedData) {
20             String key = entry.getKey();
21             reducedData.put(key, reducedData.getOrDefault(key, defaultValue: 0) + 1);
22         }
23
24         return reducedData;
25     }
26
27     Run | Debug | Run main | Debug main
28     public static void main(String[] args) {
29
30         String input = "jitendra is learning cloud jitendra is practicing mapreduce jitendra loves coding";
31
32         System.out.println(x: "Input Text:");
33         System.out.println(input);
34         System.out.println();
35
36         List<Map.Entry<String, Integer>> mapped = map(input);
37
38         System.out.println(x: "After Map:");
39         for (Map.Entry<String, Integer> entry : mapped) {
40             System.out.println("(" + entry.getKey() + ", " + entry.getValue() + ")");
41         }
42         System.out.println();
43
44         Map<String, Integer> result = reduce(mapped);
45
46         System.out.println(x: "After Reduce:");
47         for (Map.Entry<String, Integer> entry : result.entrySet()) {
48             System.out.println(entry.getKey() + " -> " + entry.getValue());
49         }
50     }
```

Output:

```
PS D:\Aayush> cd "d:\Aayush\" ; if ($?) { javac MapReduceWordCount.java };

After Map:
(aayush, 1)
(is, 1)
(learning, 1)
(cloud, 1)
(aayush, 1)
(is, 1)
(practicing, 1)
(mapreduce, 1)
(aayush, 1)
(loves, 1)
(coding, 1)

After Reduce:
cloud -> 1
coding -> 1
mapreduce -> 1
practicing -> 1
loves -> 1
aayush -> 3
is -> 2
```

Q6) Capture the packet. Capture the packet and trace the following using the appropriate methods

Wireshark is a widely-used network protocol analyzer that allows you to capture and interactively browse the traffic running on a computer network. It is an essential tool for network administrators, cybersecurity professionals, and developers for diagnosing network issues, analyzing traffic, and ensuring network security.

How Wireshark Works:

Capture: Wireshark captures network traffic using a packet capturing library like libpcap (on Unix-like systems) or WinPcap/Npcap (on Windows). It collects raw data packets from the network interface card (NIC).

Decode: Wireshark decodes the captured packets into a human-readable format. It interprets the structure of various protocols and presents the information in a structured manner.

Display: The decoded packets are displayed in a packet list pane. Clicking on a packet reveals detailed information in the packet details pane, and the raw data is shown in the packet bytes pane.

Procedure

1. Open **Wireshark.
2. Select the network interface.
3. Click Start Capturing Packets.
4. Generate traffic (open a website, ping, etc.).
5. Apply filters in the Display Filter bar.
6. Observe packets and take screenshots.

Capturing the packet

Wi-Fi						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
Apply a display filter ... <Ctrl-/>						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	2400:1a00:3b2d:915a...	2400:1a00:0:32::165	DNS	107	Standard query 0x84e1 HTTPS safebrowsing.googleapis.com
2	0.001141800	2400:1a00:3b2d:915a...	2400:1a00:0:32::165	DNS	107	Standard query 0x07c4 AAAA safebrowsing.googleapis.com
3	0.002146400	2400:1a00:3b2d:915a...	2400:1a00:0:32::165	DNS	107	Standard query 0xc069 A safebrowsing.googleapis.com
4	0.008799200	2400:1a00:0:32::165	2400:1a00:3b2d:915a...	DNS	135	Standard query response 0x07c4 AAAA safebrowsing.googleapis.com AAAA 2404:6800:4002:827::20...
5	0.008799200	2400:1a00:0:32::165	2400:1a00:3b2d:915a...	DNS	123	Standard query response 0xc069 A safebrowsing.googleapis.com A 142.250.182.106
6	0.008799200	2400:1a00:0:32::165	2400:1a00:3b2d:915a...	DNS	164	Standard query response 0x84e1 HTTPS safebrowsing.googleapis.com SOA ns1.google.com
7	0.011330500	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TCP	86	57057 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
8	0.035316500	2404:6800:4002:827::...	2400:1a00:3b2d:915a...	TCP	86	443 → 57057 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
9	0.035537700	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TCP	74	57057 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
10	0.036720700	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TCP	1486	57057 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1412 [TCP PDU reassembled in 11]
11	0.036720700	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TLSv1.3	495	Client Hello (SNI=safebrowsing.googleapis.com)
12	0.059644000	2404:6800:4002:827::...	2400:1a00:3b2d:915a...	TCP	74	443 → 57057 [ACK] Seq=1 Ack=1834 Win=267520 Len=0
13	0.060853300	2404:6800:4002:827::...	2400:1a00:3b2d:915a...	TCP	1294	[TCP Previous segment not captured] 443 → 57057 [PSH, ACK] Seq=1221 Ack=1834 Win=267520 Len=...
14	0.060956500	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TCP	86	[TCP Dup ACK 9#1] 57057 → 443 [ACK] Seq=1834 Ack=1 Win=65280 Len=0 SLE=1221 SRE=2441
15	0.061337000	2404:6800:4002:827::...	2400:1a00:3b2d:915a...	TLSv1.3	1294	[TCP Previous segment not captured] , Continuation Data
16	0.061337000	2404:6800:4002:827::...	2400:1a00:3b2d:915a...	TLSv1.3	1294	[TCP Out-Of-Order] , Server Hello
17	0.061337000	2404:6800:4002:827::...	2400:1a00:3b2d:915a...	TLSv1.3	1294	[TCP Out-Of-Order] , Continuation Data
18	0.061415300	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TCP	94	[TCP Dup ACK 9#2] 57057 → 443 [ACK] Seq=1834 Ack=1 Win=65280 Len=0 SLE=3661 SRE=4881 SLE=12...
19	0.061541900	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TCP	86	57057 → 443 [ACK] Seq=1834 Ack=2441 Win=65280 Len=0 SLE=3661 SRE=4881
20	0.061639800	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TCP	74	57057 → 443 [ACK] Seq=1834 Ack=4881 Win=65280 Len=0
21	0.061798000	2404:6800:4002:827::...	2400:1a00:3b2d:915a...	TLSv1.3	967	Continuation Data
22	0.060859300	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TLSv1.3	164	Change Cipher Spec, Application Data
23	0.060893800	2400:1a00:3b2d:915a...	2404:6800:4002:827::...	TLSv1.3	166	Application Data

> Frame 1: Packet, 107 bytes on wire (856 bits), 107 bytes captured (856 bits) on interf

> Ethernet II, Src: LiteonTechno_e3:b5:3f (d8:f3:bc:e3:b5:3f), Dst: TaicangT&MEI_15:62:2

> Internet Protocol Version 6, Src: 2400:1a00:3b2d:915a:f9da:31a4:23ae:cb2, Dst: 2400:1a

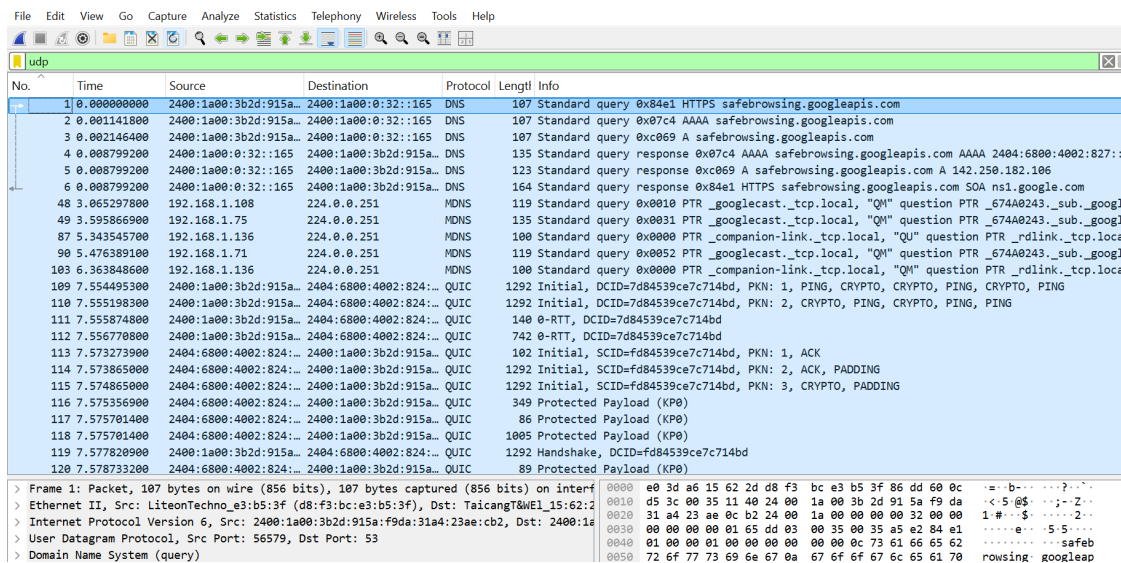
> User Datagram Protocol, Src Port: 56579, Dst Port: 53

> Domain Name System (query)

Packet that uses HTTP protocol

http						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	2400:1a00:3b2d:915a...	2400:1a00:0:32::165	HTTP	107	GET / HTTP/1.1
2	0.001141800	2400:1a00:3b2d:915a...	2400:1a00:0:32::165	HTTP	107	200 OK
3	0.002146400	2400:1a00:3b2d:915a...	2400:1a00:0:32::165	HTTP	107	200 OK

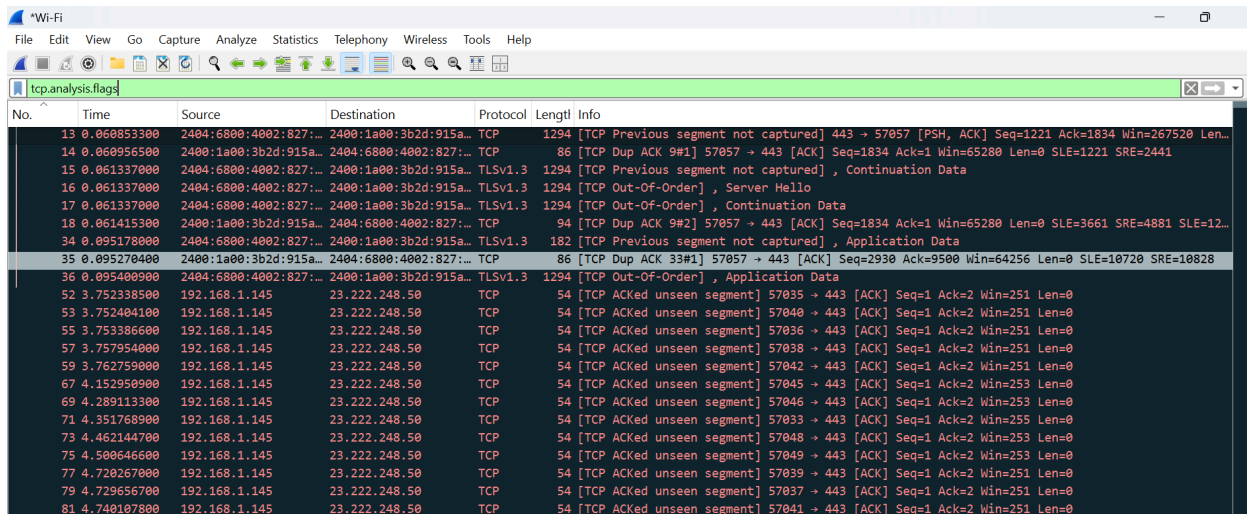
Packet that used UDP protocol



The image shows a Wireshark packet capture of a DNS query. The packet list on the left shows a series of packets, with the selected packet being a DNS query (No. 107) from 2400:1a00:3b2d:915a:: to 2400:1a00:0:32::165. The packet details pane shows the query for safebrowsing.googleapis.com. The packet bytes pane shows the raw data of the packet, including the Ethernet II header, Internet Protocol Version 6 header, and User Datagram Protocol header.

No.	Time	Source	Destination	Protocol	Length	Info
107	0.000000000	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	DNS	107	Standard query 0x84e1 HTTPS safebrowsing.googleapis.com

Packet that are suspicious



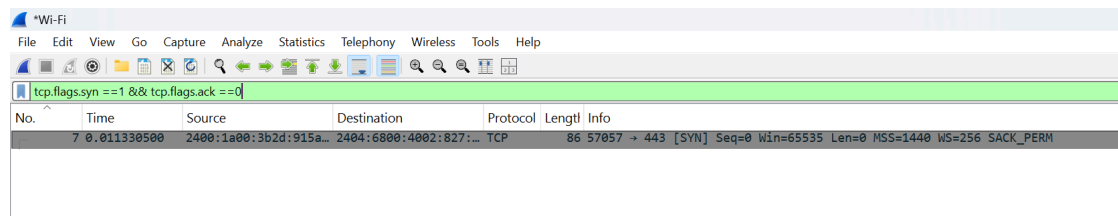
The image shows a Wireshark packet capture of a series of TCP and TLSv1.3 packets. The packet list on the left shows a series of packets, with the selected packet being a TCP packet (No. 13) from 2400:1a00:3b2d:915a:: to 2400:1a00:0:32::165. The packet details pane shows the TCP header and the TLSv1.3 header. The packet bytes pane shows the raw data of the packet, including the Ethernet II header, Internet Protocol Version 6 header, and User Datagram Protocol header.

No.	Time	Source	Destination	Protocol	Length	Info
13	0.060853300	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TCP	1294	[TCP Previous segment not captured] 443 → 57057 [PSH, ACK] Seq=1221 Ack=1834 Win=267520 Len=0
14	0.060956500	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TCP	86	[TCP Dup ACK 9#1] 57057 → 443 [ACK] Seq=1834 Ack=1 Win=65280 Len=0 SLE=1221 SRE=2441
15	0.061337000	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TLSv1.3	1294	[TCP Previous segment not captured] , Continuation Data
16	0.061337000	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TLSv1.3	1294	[TCP Out-Of-Order] , Server Hello
17	0.061337000	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TLSv1.3	1294	[TCP Out-Of-Order] , Continuation Data
18	0.061415300	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TCP	94	[TCP Dup ACK 9#2] 57057 → 443 [ACK] Seq=1834 Ack=1 Win=65280 Len=0 SLE=3661 SRE=4881 SLE=12...
34	0.095178000	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TLSv1.3	182	[TCP Previous segment not captured] , Application Data
35	0.095270400	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TCP	86	[TCP Dup ACK 33#1] 57057 → 443 [ACK] Seq=2930 Ack=9500 Win=64256 Len=0 SLE=10720 SRE=10828
36	0.095400900	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TLSv1.3	1294	[TCP Out-Of-Order] , Application Data
52	3.752338500	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57035 → 443 [ACK] Seq=1 Ack=2 Win=251 Len=0
53	3.752404100	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57040 → 443 [ACK] Seq=1 Ack=2 Win=251 Len=0
55	3.753386600	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57036 → 443 [ACK] Seq=1 Ack=2 Win=251 Len=0
57	3.757954000	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57038 → 443 [ACK] Seq=1 Ack=2 Win=251 Len=0
59	3.762759000	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57042 → 443 [ACK] Seq=1 Ack=2 Win=251 Len=0
67	4.152950900	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57045 → 443 [ACK] Seq=1 Ack=2 Win=253 Len=0
69	4.289113300	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57046 → 443 [ACK] Seq=1 Ack=2 Win=253 Len=0
71	4.351768900	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57033 → 443 [ACK] Seq=1 Ack=2 Win=255 Len=0
73	4.462144700	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57048 → 443 [ACK] Seq=1 Ack=2 Win=253 Len=0
75	4.500646600	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57049 → 443 [ACK] Seq=1 Ack=2 Win=253 Len=0
77	4.720627000	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57039 → 443 [ACK] Seq=1 Ack=2 Win=251 Len=0
79	4.729656700	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57037 → 443 [ACK] Seq=1 Ack=2 Win=251 Len=0
81	4.740107800	192.168.1.145	23.222.248.50	TCP	54	[TCP ACKed unseen segment] 57041 → 443 [ACK] Seq=1 Ack=2 Win=251 Len=0

Packets for TCP handshaking

Filter for SYN

Tcp.flags.syn == 1 && tcp.flags.ack == 0



The image shows a Wireshark packet capture of a SYN packet. The packet list on the left shows a series of packets, with the selected packet being a SYN packet (No. 7) from 2400:1a00:3b2d:915a:: to 2400:1a00:0:32::165. The packet details pane shows the TCP header and the SYN flag. The packet bytes pane shows the raw data of the packet, including the Ethernet II header, Internet Protocol Version 6 header, and User Datagram Protocol header.

No.	Time	Source	Destination	Protocol	Length	Info
7	0.011330500	2400:1a00:3b2d:915a::	2400:1a00:0:32::165	TCP	86	57057 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM

Filter for SYN-ACK

```
tcp.flags.syn == 1 && tcp.flags.ack == 1
```

The image shows a Wireshark packet capture. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. Below the menu is a toolbar with various icons for packet capture and analysis. The packet list pane shows a single packet, number 8, at time 0.035316500, from source 2404:6800:4002:827::... to destination 2400:1a00:3b2d:915a::... on port 443, with protocol TCP. The packet details pane shows the TCP header with flags SYN, ACK, Seq=0, Ack=1, Win=65535, Len=0, MSS=1412, SACK_PERM, and WS=256. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
8	0.035316500	2404:6800:4002:827::...	2400:1a00:3b2d:915a::...	TCP	86	443 → 57057 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256

Filter for ACK

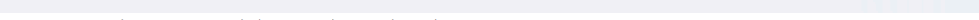
```
tcp.flags.ack == 1 && tcp.flags.syn == 0
```

Wi-Fi

FileEditViewGoCaptureAnalyzeStatisticsTelephonyWirelessToolsHelp

Packet with syn and ack flag both

```
Tcp.flags.syn == 1 && tcp.flags.ack == 1
```



The image shows the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The toolbar contains icons for various functions like opening files, capturing, and analyzing. The main display area is divided into three panes: Packet List, Packet Details, and Packet Bytes. The Packet List pane shows a single packet (No. 0) from 10.0.3.15 to 10.0.1.1 on port 80. The Packet Details pane shows the TCP header with the SYN flag set. The Packet Bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
0	0.035316500	2404:6800:4002:827::...	2400:1a00:3b2d:915a::...	TCP	86	443 → 57057 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256

Q4) Perform security drills and audit using Nmap

This lab focuses on understanding and assessing the security posture of networks is crucial for safeguarding sensitive information and maintaining operational integrity. As part of this lab exercise, we aim to dive into the security status of two distinct networks: MBM College and Google. This investigation will utilize various specialized tools to perform comprehensive scans and evaluations, providing insights into potential vulnerabilities, configuration weaknesses, and overall network security readiness.

Objectives

The objectives of this lab are as follows:

- Identify and map the network infrastructure of MBM College and Google using network scanning tools.
- Conduct comprehensive vulnerability assessments to detect potential security weaknesses within the networks.
- Perform web application security assessments on the web servers hosted by MBM College and Google to identify common vulnerabilities and misconfigurations.
- Document and report findings, including prioritized vulnerabilities and recommended mitigation strategies for each network.

Opened kali linux and terminal installed nmap package. Then different security drills and audits were performed. Following are the security drills and audits performed

1. Hosted discovery scan (Ping sweep)

This identifies which devices are "alive" without checking individual ports. I used command "sudo nmap -sn -oN discovery_scan.txt google.com"

```
(kali㉿kali)-[~]
└─$ sudo nmap -sn -oN discovery_scan.txt google.com
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-11 05:53 -0400
Nmap scan report for google.com (142.250.192.238)
Host is up (0.019s latency).
Other addresses for google.com (not scanned): 2404:6800:4002:808::200e
rDNS record for 142.250.192.238: tzdela-ax-in-f14.1e100.net
Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds

(kali㉿kali)-[~]
└─$
```

2. Stealth SYN Scan (Common Ports)

This is the standard "silent" scan. it checks the 1,000 most common ports. I used command "sudo nmap -sS -T4 -oN stealth_scan.txt google.com"

```
(kali㉿kali)-[~]
└─$ sudo nmap -sS -T4 -oN stealth_scan.txt google.com
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-11 05:56 -0400
Nmap scan report for google.com (142.250.192.238)
Host is up (0.012s latency).
Other addresses for google.com (not scanned): 2404:6800:4002:811::200e
rDNS record for 142.250.192.238: tzdela-ax-in-f14.1e100.net
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
113/tcp    closed ident
443/tcp    open  https
2000/tcp   open  cisco-sccp
5060/tcp   open  sip

Nmap done: 1 IP address (1 host up) scanned in 4.91 seconds
```

3. Service Version & OS Detection

This provides the "Audit" data by identifying the software versions and the operating system. I used command "sudo nmap -sY -o -oN audit_scan.txt [google.com](https://www.google.com)"

```

(kali@kali)-[~]
└─$ sudo nmap -oN audit_scan.txt google.com
Starting Nmap 7.90 ( https://nmap.org ) at 2026-03-11 05:58 -0400
Stats: 0:01:29 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 06:01 (0:01:24 remaining)
Stats: 0:01:31 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 06:03 (0:01:26 remaining)
Stats: 0:02:06 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 06:02 (0:02:01 remaining)
Stats: 0:02:36 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 06:03 (0:02:31 remaining)
Stats: 0:02:50 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 0.00% done
Nmap scan report for google.com (142.250.192.238)
Host is up (0.016s latency).
Other addresses for google.com (not scanned): 2404:6800:4002:811::200e
rDNS record for 142.250.192.238: t2dela-ax-in-f14.1e100.net
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      gws
113/tcp   closed ident
443/tcp   open  ssl/https gws
2000/tcp  open  cisco-sccp?
5060/tcp  open  sip?
2 services unrecognized despite returning data. If you know the service/version, please submit the following fingerprints at https://nmap.org/cgi-bin/submit.cgi?new-service :
=====
NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)
=====
SF:Port80-TCP:V=7.98X=7ND=3/11KTime=69813CD2NP=x86_64-pc-linux-gnuXr(GetR
SF:request,A565,"HTTP/1.1.0x20200x200K\r\nDate:\x20Wed,\x2011\x20Mar\x2020
SF:20\x2009:58:41\x20GMT\r\nExpires:\x20-1\r\nCache-Control:\x20private,\x
SF:20max-age=0\r\nContent-Type:\x20text/html;\x20charset=ISO-8859-1\r\nCon
SF:tent-Security-Policy:Report-Only;\x20nojsct=arc\x20none"base-uri\x20"
SF:self"jsrict-src\x20"nonce=ctsv30R905vhQNDNMeAqDA"\x20"strict-dynamic"\
SF:x20"report-sample"\x20"unsafe-eval"\x20"unsafe-inline"\x20https:\x20htt
SF:ps:report-uri\x20https://csp.withgoogle.com/csp/gws/other-hp\r\nRepor
SF:tting-Endpoints:\x20default"/www.google.com/httpservice/retry/jserr
SF:on/Telnet/yabeyVLM61s0pMSDFgAYvcadncasherror=Page20crashshjsel3dbver
SF:=23946dpf=MDfC1vcF4B73Cw4R0B5-urVHOOPDI29AHAFKwdbko"\r\nP3P:\x20CP=
SF:"This\x20is\x20not\x20a\x20P3P\x20policy!\x20See\x20g.co/p3phelp\x20fo
SF:r:\x20more\x20info.\r\nServer:\x20gws\r\nX-XSS-Protection:\x200\r\nX-
SF:Frame-Options:\x20SAMEORIGIN\r\nSet-Cookie:\x20_Secure=STRP-AEEP7gLSst
SF:yGKr06DIx7855Vx0a82NQZ21jvxDkaAt5Dbvc-VHTLoITNz7Q08PMM5xann8h0rbKICem1
SF:eCO4qyy78RmK3JdxEXK;\x20expires=Wed,\x2011-Mar-2026\x2010:03:41\x20GMT;
SF:\x20path=/;\x20domain=\r\n(HTTPOptions,60C,"HTTP/1.1.0x20485\x20Method\x2
SF:0Not\x20Allowed\r\nContent-Type:\x20text/html;\x20charset=UTF-8\r\nNrefe
SF:errer-Policy:\x20no-referrer\r\nContent-Length:\x201592\r\nDate:\x20Wed,
SF:\x2011\x20Mar\x202026\x2009:58:42\x20GMT\r\n\r\nnc100CTYPE\x20html>\ncht

```

4. Vulnerability Audit (Scripting Engine)

This uses the Nmap Scripting Engine (NSE) to check for known security flaws (CVEs). I used command “sudo nmap -sV --script vuln -oN vulnerability_report.txt google.com”

```

(kali@kali)-[~]
└─$ sudo nmap -sV --script vuln -oN vulnerability_report.txt google.com
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-11 06:05 -0400
Stats: 0:00:18 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
Stats: 0:00:28 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
Stats: 0:01:00 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
Stats: 0:01:50 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 06:08 (0:01:39 remaining)
Stats: 0:03:16 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 86.52% done; ETC: 06:08 (0:00:04 remaining)
Stats: 0:04:11 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 98.70% done; ETC: 06:09 (0:00:01 remaining)
Stats: 0:05:10 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 98.70% done; ETC: 06:10 (0:00:02 remaining)
Stats: 0:05:39 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 98.70% done; ETC: 06:10 (0:00:02 remaining)
Stats: 0:05:41 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 98.70% done; ETC: 06:10 (0:00:02 remaining)

```